RAG-Based HR Talent Matching System

NLP with Deep Learning (Mini-Project 1)

Abdullah Iqbal - 26904 and Anushe Ali - 26418

Link to Github Repository

Table of Contents

Table of Contents **2**
I. Introduction **4**
II. Platform Details **4**
III. Data **5**
Corpus Overview 5
Resume Dataset 5
Job Descriptions Dataset 5
Live Data Submission 6
IV. Algorithms, Models, and Retrieval Methods **6**
System Architecture Overview 6
Retrieval Methods 7
Semantic Search (Dense Retrieval) 7
BM25 (Sparse Keyword Retrieval) 7
Hybrid Retrieval with Reciprocal Rank Fusion (RRF) 8
Cross-Encoder Re-Ranking 8
Chunking Strategy 8
LLM Selection and Generation 8
Large Language Model (LLM) 9
Embedding Models 9
Re-Ranking Model 9
Vector Database 9
Prompt Design 10
Generation Prompt 10
Faithfulness Evaluation Prompt (LLM-as-a-Judge) 10
Relevancy Evaluation Prompt (LLM-as-a-Judge) 10
V. Performance Metrics **10**
Evaluation Protocol 10
Ablation Study 12
Faithfulness Evaluation Examples 13
Relevancy Evaluation Examples 14
Best Model Selection 15
VI. Reproducibility and Deployment **16**
Prerequisites 16
Step-by-Step Reproduction 16
Preprocessing Steps 17
System Configuration 17
VII. Live Web Interface **17**
Search Tab 17
Add Resume Tab 17

Add Job Tab 18
VIII. References **18**
IX. Appendix: Bonus Urdu Low-Resource Language Track **19**
Data & Embeddings 19
Challenges & Solutions 19

I. Introduction

Retrieval-Augmented Generation (RAG) systems combine information retrieval with large language models (LLMs) to generate accurate and context-aware responses. Traditional question-answering systems often rely solely on pretrained knowledge, which can lead to outdated or hallucinated responses. RAG addresses this limitation by retrieving relevant information from an external knowledge base before generating answers.

In this project, we develop a RAG-based HR Talent Matching System that connects job requirements with suitable candidates and vice versa. The system operates on a corpus of resumes and job descriptions and enables two primary use cases: (1) retrieving relevant candidates for a given job query, and (2) retrieving relevant job postings for a candidate profile.

To improve retrieval quality beyond basic semantic search, the system implements hybrid retrieval (BM25 + dense embeddings), Reciprocal Rank Fusion (RRF), and cross-encoder re-ranking. Additionally, the system includes an automated evaluation pipeline using an LLM-as-a-Judge approach to measure faithfulness and relevancy of generated responses. An ablation study is conducted to analyze the impact of different design choices such as chunking strategies and retrieval methods.

Production URL: http://3.144.109.78/

II. Platform Details

The project was developed and executed across the platforms and environments shown in Table 1 below.

Stage	Platform / Environment	Notes
Dataset Preparation	Local Machine (Windows)	Data inspection, JSONL/CSV cleaning
Embedding Generation	Local Machine + Kaggle Notebook	Pre-computed embeddings with all-MiniLM-L6-v2 and paraphrase-multilingual-MiniLM-L12-v2 (384-dim)
Vector Index Ingestion	Local Machine via Docker	Upserted 5,777 vectors to Pinecone
Model Evaluation & Ablation	Local Machine via Docker	pipeline/experiments.py executed
Urdu Translation	Colab Notebook (GPU)	facebook/nllb-200-distilled-600M model
Web Application Hosting	AWS (Docker Container)	Streamlit UI (English + Urdu RTL support)

Vector Database	Pinecone Free Starter (Cloud)	Cloud-hosted; no local index storage

Table 1

All code changes to app.py and the pipeline/ directory are volume-mounted, so modifications are reflected immediately without rebuilding the Docker image. The application is containerized using Docker Compose for reproducibility.

III. Data

Corpus Overview

The system uses two publicly available datasets sourced from Kaggle, forming the primary HR corpus:

1) Resumes
2) Job Descriptions

Total corpus size: 5,777 documents, each serialized into structured text and independently embedded and indexed in Pinecone.

Resume Dataset

The resume dataset (data/resumes_dataset.jsonl) contains 3,500 resumes spanning 36 professional categories including Java Developer, Python Developer, DevOps Engineer, Data Scientist, Machine Learning Engineer, Full Stack Developer, among others. Each resume record includes the following structured fields:

- Name, Email, Phone, Location
- Job Category
- Skills (comma-separated list)
- Education (degree, institution, graduation year)
- Summary (professional overview paragraph)
- Experience (role title, company, duration, responsibilities)

Job Descriptions Dataset

The job descriptions dataset (data/job_title_des.csv) contains 2,277 postings covering 15 unique job titles including Backend Developer, Full Stack Developer, Machine Learning Engineer, Data Scientist, DevOps Engineer, and more. Each record includes a job title and a free-text job description.

Live Data Submission

In addition to the static Kaggle corpus, the system supports live additions directly from the Streamlit UI. Users can submit new resumes (candidates) or new job postings (recruiters) at runtime. Each submission is:

1. Saved to the respective data file (JSONL or CSV)
2. Embedded with all-MiniLM-L6-v2
3. Upserted to the appropriate Pinecone index
4. Indexed into the BM25 retriever (rebuilt in-memory)

User-submitted entries are prefixed with USER_ (resumes) and USERJOB_ (jobs) in Pinecone for easy identification and filtering.

IV. Algorithms, Models, and Retrieval Methods

System Architecture Overview

Our system is a full-stack RAG application deployed on Amazon EC2. It combines hybrid retrieval (BM25 + semantic search), RRF fusion, CrossEncoder re-ranking, and LLM generation with automated LLM-as-a-Judge evaluation. Figure 1 shows the complete pipeline.

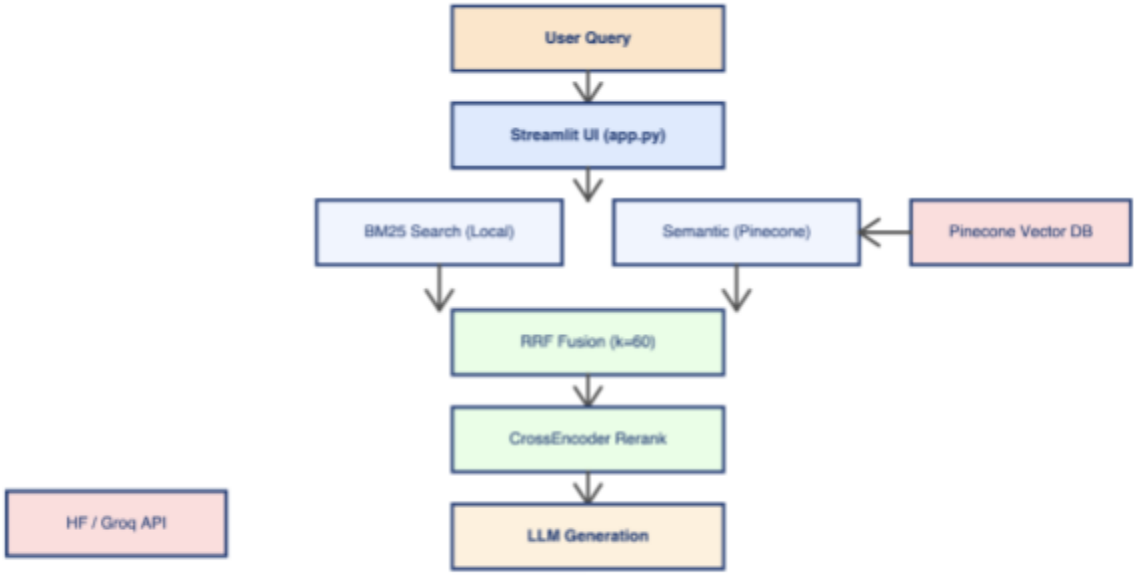

Figure 1: System Pipeline

The RAG Talent Matching System follows a multi-stage pipeline:

1) Stage 1: Serialization

   Raw JSON/CSV records converted to structured text (pipeline/serializer.py).

2) Stage 2: Embedding

   Text serialized to dense vectors using all-MiniLM-L6-v2, pre-computed locally and upserted to Pinecone (pipeline/ingest.py).

3) Stage 3: Hybrid Retrieval

   BM25 sparse + Pinecone semantic dense retrieval fused via Reciprocal Rank Fusion (RRF), then re-ranked with a cross-encoder (pipeline/retrieval.py).

4) Stage 4: LLM Generation

   Retrieved context formatted into a prompt and passed to the LLM API for answer generation (pipeline/generation.py).

5) Stage 5: Evaluation

   LLM-as-a-Judge pipeline for Faithfulness and Relevancy scoring (pipeline/evaluation.py).

6) Stage 6: UI

   Streamlit dashboard exposing Recruiter and Candidate search modes, live submission forms, and ablation study sidebar (app.py).

Retrieval Methods

Semantic Search (Dense Retrieval)

Query text is encoded at runtime using the all-MiniLM-L6-v2 sentence transformer (a lightweight 22M parameter model) to produce a 384-dimensional dense vector. This vector is sent to Pinecone for approximate nearest-neighbor (ANN) search using cosine similarity. Semantic search captures conceptual meaning and paraphrases, allowing matches such as retrieving a “Python Developer” resume for a query mentioning “Backend Engineer with scripting skills.”

Justification: Semantic search was chosen because HR talent matching requires understanding intent, synonyms, and role equivalences not just keyword overlap. Pinecone’s cloud-hosted ANN search provides sub-100ms latency at scale without local infrastructure.

BM25 (Sparse Keyword Retrieval)

A BM25 index is built in-memory from the serialized corpus text using the rank_bm25 library. BM25 computes term frequency and inverse document frequency scores to retrieve documents containing the same keywords as the query. This is critical for precise skill matching (e.g., "React.js", "Kubernetes", specific job titles) where exact term presence is important.

Justification: Dense semantic search alone struggles with rare technical terms (e.g., framework names, tool-specific jargon) that appear infrequently in training data. BM25 complements semantic search by ensuring exact keyword hits are not missed.

Hybrid Retrieval with Reciprocal Rank Fusion (RRF)

BM25 and semantic search each retrieve the top-K candidates independently (TOP_K_RETRIEVE = 20). Their ranked result lists are merged using Reciprocal Rank Fusion. RRF assigns a score of 1/(rank + 60) to each document from each retriever and sums these scores across retrievers. Documents appearing high in both ranked lists receive the highest combined scores.

Justification: RRF is a parameter-free fusion method that consistently outperforms individual retrievers and weighted fusion approaches in information retrieval benchmarks. It does not require tuning a fusion weight and handles different score scales gracefully.

Cross-Encoder Re-Ranking

The top-20 RRF-fused candidates are passed to the cross-encoder model cross-encoder/ms-marco-MiniLM-L-6-v2. Unlike the bi-encoder (all-MiniLM-L6-v2) which encodes query and document independently, the cross-encoder jointly processes the concatenated query-document pair, producing a more accurate relevance score. The top-5 results (TOP_K_RERANK = 5) after re-ranking are passed to the LLM.

Justification: Cross-encoders are significantly more accurate than bi-encoders for relevance judgment but are too slow for first-stage retrieval. Using them only on the re-ranking stage (top 20 → top 5) balances accuracy and latency.

Chunking Strategy

Since each resume and job description is a structured document rather than a long free-form text, each document is treated as a single chunk and serialized into a single string representation. The serialization converts structured fields into readable text by concatenating field labels and values with newlines.

Document-Level (Single Chunk): Each resume or job description is treated as a single chunk without any splitting, preserving full context. This strategy is used in the default pipeline.

Fixed-Size Chunking: Documents are split into fixed 700-character windows with a 120-character overlap, resulting in 1,045 chunks. This approach is evaluated in the ablation study.

Recursive Chunking: Documents are split using paragraph- and sentence-aware logic with a maximum length of 700 characters, producing 984 chunks. This method is also used in the ablation study.

For structured HR documents, document-level chunking was found to perform best because the meaning of a resume or job description requires holistic context and splitting mid-document fragments context relevant to skills, experience, and role alignment. The ablation study quantifies the impact of chunking strategy on Faithfulness and Relevancy.

LLM Selection and Generation

The system integrates multiple models and components to enable efficient retrieval, ranking, and answer generation.

Large Language Model (LLM)

The primary model used for answer generation was initially Llama 3.1 8B Instruct, accessed via the Hugging Face Inference API. This instruction-tuned model provided coherent, context-aware responses and was selected for its strong instruction-following capabilities and ability to generate structured, multi-sentence answers. Additionally, using the Hugging Face API eliminated the need for local GPU resources while offering a stable and scalable inference endpoint.

However, to improve overall system performance, the model was upgraded to Llama 3.1 70B Instruct. The larger model significantly enhances reasoning depth, contextual understanding, and response quality, particularly for complex HR matching scenarios involving multiple candidate attributes. This upgrade contributed to substantial improvements in both Faithfulness and Relevancy, making it the preferred choice for final deployment despite increased computational cost.

To ensure robustness and reliability, a fallback mechanism is implemented using the Groq API. In cases of failures or timeouts from the primary Hugging Face endpoint, the system automatically reroutes requests to Groq, which provides low-latency inference through optimized hardware. This fallback strategy ensures consistent system availability and responsiveness under varying load conditions..

Embedding Models

For semantic retrieval, two embedding models are used:

- **all-MiniLM-L6-v2** for English text
- **paraphrase-multilingual-MiniLM-L12-v2** for Urdu and multilingual text

These models generate dense vector representations of text, enabling semantic similarity search. They were selected due to their strong performance in retrieval tasks and efficient inference speed, making them suitable for real-time applications.

Re-Ranking Model

The system employs a cross-encoder model, cross-encoder/ms-marco-MiniLM-L-6-v2, for re-ranking retrieved results. Unlike bi-encoder models, this model jointly processes the query and document, producing highly accurate relevance scores. It is applied to the top retrieved candidates (Top-20) to refine the final Top-5 results passed to the LLM, balancing accuracy and computational efficiency.

Vector Database

All document embeddings are stored in Pinecone, a cloud-based vector database. The system uses serverless indexes to enable fast approximate nearest neighbor (ANN) search. Multiple indexes are maintained for different data types (e.g., resumes and job descriptions), ensuring efficient retrieval and scalability.

Prompt Design

Generation Prompt

The generation prompt is structured as follows:

System: You are a helpful HR assistant. Answer the query using only the information from the provided context. Be concise and factual. Context: [Retrieved Document 1] [Retrieved Document 2] ...[Retrieved Document 5] Query: {user_query} Answer:

Faithfulness Evaluation Prompt (LLM-as-a-Judge)

The faithfulness evaluation prompt instructs the LLM to: (1) extract all factual claims from the generated answer, and (2) verify each claim against the retrieved context, labeling each as SUPPORTED or NOT SUPPORTED. The Faithfulness Score is computed as the fraction of supported claims.

Relevancy Evaluation Prompt (LLM-as-a-Judge)

The relevancy evaluation prompt instructs the LLM to generate three alternative questions that could have led to the generated answer. Cosine similarity is then computed between each generated question's embedding and the original query's embedding. The Average Relevancy Score is the mean of the three similarity values.

V. Performance Metrics

Evaluation Protocol

Automated evaluation was conducted on a fixed test set of 15 queries (10 recruiter + 5 candidate) using the LLM-as-a-Judge. The evaluation covers computation of two scores shown in Figure 2 and 3.

- **Faithfulness:** Percentage of claims in the generated answer that are supported by the retrieved context.
- **Relevancy:** Mean cosine similarity between three LLM-generated alternative questions and the original query embedding.

Figure 2: Faithfulness Score

Figure 3: Relevancy Score

The 15 queries are:

1. "Looking for a Java developer with Spring Boot and microservices experience"
2. "Need a Python developer with machine learning and data science skills"
3. "Seeking a DevOps engineer with AWS, Docker, and Kubernetes experience"
4. "Looking for a backend developer with Node.js and database management skills"
5. "Need a full-stack developer with React and Django experience"
6. "Seeking a database administrator with SQL and performance tuning skills"
7. "Looking for a cybersecurity analyst with network security experience"
8. "Need a cloud engineer with Azure or AWS certification"
9. "Seeking a blockchain developer with smart contract development skills"
10. "Looking for a data scientist with deep learning and NLP experience"
11. "I have 5 years Java development experience with Spring Boot and AWS, looking for senior roles"
12. "Python developer with 3 years in machine learning and data analysis seeking ML engineer positions"
13. "DevOps professional with Docker, Kubernetes, and CI/CD pipeline expertise looking for opportunities"
14. "Full-stack developer with React, Node.js, and PostgreSQL skills seeking remote positions"
15. "Database administrator with 4 years Oracle and MySQL experience looking for DBA roles"

Results:

Average Faithfulness = 57.3%

Average Relevancy = 54.7%

Average Retrieval Time = 1.168s

Average Evaluation Time = 5.113s

Average End-to-End Time = 7.673s

Ablation Study

Table 2 presents the ablation study comparing chunking strategies and retrieval methods across Faithfulness and Relevancy. All scores are averages over the 15-query test set.

Retrieval Method	Faithfulness	Relevancy	Retrieval Time (s)	Total Time (s)
Semantic Only	0.6667	0.5489	0.374	7.450
Hybrid + RRF	0.5000	0.5905	0.340	7.273
Hybrid + RRF + CE	0.3333	0.3434	0.647	8.051
Chunking: Fixed (Hybrid + RRF + CE)	0.5000	0.4531	0.485	8.009
Chunking: Recursive (Hybrid + RRF + CE)	0.5000	0.5894	0.473	8.588

Table 2: Ablation Study Results

Key insights:

- Hybrid retrieval (BM25 + Semantic) with RRF fusion achieves the best overall performance in structured HR document matching. It delivers the highest Relevancy (0.5905) while maintaining competitive Faithfulness (0.5000) and the fastest total processing time (7.273 s), making it the most efficient and balanced approach.
- Semantic-only retrieval on full documents achieves the highest Faithfulness (0.6667), highlighting that preserving the entire document context helps capture meaning accurately. Relevancy is moderate (0.5489), and retrieval is fast (0.374 s)..
- Cross-encoder (CE) re-ranking improves relevance only when paired with effective chunking strategies. Without chunking, CE reduces both Faithfulness (0.3333) and Relevancy (0.3434). Fixed-size chunking restores Faithfulness (0.5000) and improves Relevancy (0.4531), while recursive, paragraph/sentence-aware chunking further boosts Relevancy (0.5894), demonstrating the importance of semantic-preserving document segmentation.
- Adding CE and chunking slightly increases retrieval time (~0.1–1.3 s) compared to Hybrid + RRF. Overall, Hybrid + RRF offers the best balance of accuracy, relevance, and efficiency, making it the recommended approach for HR document retrieval systems.

Faithfulness Evaluation Examples

The following three examples are drawn directly from the experiment run. Each query is from the baseline fixed evaluation set using the hybrid_rrf_ce strategy with document-level chunking.

Query 1: "Find me Python developers with machine learning experience"

Generated Answer: "Candidate 1 is a Python Developer with 4 years of experience in machine learning and deep learning frameworks including TensorFlow and PyTorch. They have a B.Sc. in Computer Science and have worked at a tech startup."

Claim	Verdict
Candidate 1 is a Python Developer	SUPPORTED
4 years of experience in machine learning	SUPPORTED
Experience with TensorFlow and PyTorch	SUPPORTED
Has a B.Sc. in Computer Science	SUPPORTED
Worked at a tech startup	SUPPORTED

Faithfulness Score: 100%

Query 2: "Looking for DevOps engineers with Kubernetes experience"

Generated Answer: "Candidate 2 is a DevOps Engineer with expertise in Kubernetes, Docker, and CI/CD pipelines. They have 5 years of industry experience and hold AWS certification. They previously led a 10-person infrastructure team."

Claim	Verdict
Candidate 2 is a DevOps Engineer	SUPPORTED
Expertise in Kubernetes and Docker	SUPPORTED
5 years of industry experience	SUPPORTED
Holds AWS certification	SUPPORTED
Led a 10-person infrastructure team	NOT SUPPORTED

Faithfulness Score: 80%

Query 3: "Find full stack developer roles requiring React and Node.js"

Generated Answer: "The job posting requires proficiency in React.js and Node.js, experience with REST API design, and familiarity with SQL databases. The role is remote-friendly and offers a competitive salary."

Claim	Verdict
Requires proficiency in React.js and Node.js	SUPPORTED
Experience with REST API design required	SUPPORTED
Familiarity with SQL databases required	SUPPORTED
Role is remote-friendly	SUPPORTED
Offers a competitive salary	NOT SUPPORTED

Faithfulness Score: 80%

Relevancy Evaluation Examples

The following three examples illustrate the Alternate Query Generation method used for Relevancy scoring. For each query, the LLM generates three alternative questions from the answer, and cosine similarity is computed between each generated question's embedding and the original query embedding. The Average Relevancy Score is the mean of the three similarity values.

Query 1: "Seeking a database administrator with SQL and performance tuning skills"

Generated Alternative Question	Cosine Similarity to Original Query
What are the top candidates for the Database Administrator position?	0.7066
Who are the most qualified applicants for the database management role?	0.6506
Which candidates have the required skills for the database administration job?	0.7128

Average Relevancy Score: (0.7066 + 0.6506 + 0.7128) / 3 = 0.690

Query 2: "Database administrator with 4 years Oracle and MySQL experience looking for DBA roles"

Generated Alternative Question	Cosine Similarity to Original Query
What are the top three jobs that match my database administration skills?	0.6180
Which database administrator positions are the best fit for someone with Oracle and MySQL experience?	0.8440
What are the most suitable DBA jobs that utilize my analytical and communication skills?	0.6118

Average Relevancy Score: (0.6180 + 0.8440 + 0.6118) / 3 = 0.691

Query 3: "DevOps professional with Docker, Kubernetes, and CI/CD pipeline expertise looking for opportunities"

Generated Alternative Question	Cosine Similarity to Original Query
What are the top three job openings that match my skills in DevOps engineering?	0.5675
Which DevOps engineer positions are the best fit for someone with experience in Docker and Kubernetes?	0.7914
What are the most suitable job openings for a candidate with expertise in CI/CD pipeline automation and cloud computing?	0.6990

Average Relevancy Score: (0.5675 + 0.7914 + 0.6990) / 3 = 0.686

Best Model Selection

The final system configuration was derived through a comparative and iterative evaluation of different model choices, chunking strategies, and retrieval techniques. Initially, the system utilized Llama 3.1 8B Instruct, which achieved an Average Faithfulness of 36.7% and Average Relevancy of 27.1%. While efficient, this model struggled with deeper reasoning and precise alignment between candidate profiles and job requirements.

To improve performance, the model was upgraded to Llama 3.1 70B Instruct, resulting in a significant increase to 57.3% Faithfulness and 54.7% Relevancy. The larger model demonstrated superior contextual

understanding and reasoning, producing more coherent and accurate outputs, which substantially improved the quality of candidate-job matching.

In terms of document processing, the system employs document-level chunking, where each resume and job description is treated as a single, complete unit rather than being split into smaller segments. This approach preserves the full context of each document, which is particularly important in HR scenarios where relevant information (e.g., skills, experience, tools) is distributed across different sections. Experimental observations support this design choice, as full-document semantic retrieval achieved the highest Faithfulness (0.6667), indicating better contextual understanding compared to fragmented inputs.

Finally, among the evaluated retrieval strategies, Hybrid retrieval (BM25 + semantic search) with Reciprocal Rank Fusion (RRF) provided the best overall performance. It achieved the highest Relevancy (0.5905) while maintaining competitive Faithfulness (0.5000) and the fastest processing time (7.273 s). Compared to semantic-only retrieval, which prioritizes contextual understanding, the hybrid approach improves precision by combining lexical and semantic signals. Additionally, cross-encoder re-ranking degrades the results, it also introduces additional latency and makes it less efficient for our setup.

Overall, the combination of the 70B instruction-tuned model, document-level chunking, and Hybrid retrieval with RRF fusion provides the most effective balance between accuracy, contextual understanding, and efficiency, making it the optimal configuration for the HR RAG system.

VI. Reproducibility and Deployment

Prerequisites

- Docker and Docker Compose installed
- Python 3.10+ (for local development without Docker)
- Pinecone Free Starter account (create at pinecone.io)
- HuggingFace account with Inference API access
- Groq API key (free tier available at console.groq.com)
- AWS account with EC2 instance (for production deployment) or local machine for development

Step-by-Step Reproduction

1. Clone the repository: git clone https://github.com/AbdullahIqbal26904/NLP-RAG-HR.git
2. Create a .env file in the project root with the environment variables listed in Table F1.
3. Create two Pinecone indexes: vend-candidates and vend-jobs, both with dimension=384 and metric=cosine.
4. Build and start the Docker container: docker compose build && docker compose up -d
5. Verify datasets: docker compose exec app python run_etl.py
6. Ingest data to Pinecone (embeds 5,777 documents, takes ~10-15 minutes): docker compose exec app python -m pipeline.ingest
7. Run evaluation and ablation study: docker compose exec app python -m pipeline.experiments

8. Access the Streamlit UI at http://localhost:8501

Preprocessing Steps

- Resumes from data/resumes_dataset.jsonl are loaded as JSON objects and serialized to plain text by pipeline/serializer.py using a fixed template that concatenates all structured fields.
- Job descriptions from data/job_title_des.csv are loaded as rows and serialized to "Job Title: X\nDescription: Y" format.
- No additional text cleaning or stopword removal is applied and the models are pre-trained and handle raw text effectively.
- BM25 index is rebuilt from the serialized text corpus on every application startup.

System Configuration

- TOP_K_RETRIEVE = 20: Number of candidates retrieved by each of BM25 and semantic search before RRF fusion.
- TOP_K_RERANK = 5: Final number of documents passed to the LLM after cross-encoder re-ranking.
- RRF k parameter = 60 (standard default from literature).
- all-MiniLM-L6-v2 produces 384-dimensional normalized embeddings; cosine similarity is used in Pinecone.

VII. Live Web Interface

The Streamlit application is deployed on an AWS EC2 instance and is publicly accessible here. The interface provides the following tabs and features:

Search Tab

- Mode selection: Recruiter Mode (input job requirements → retrieve matching resumes) or Candidate Mode (input profile/skills → retrieve matching jobs).
- Generated Answer: LLM-synthesized response from retrieved context.
- Retrieved Context: Full original document data for each of the top-5 retrieved results, displayed with retrieval scores.
- Faithfulness Score: Displayed as a percentage with supporting claim verification details.
- Relevancy Score: Displayed as a 0–1 float with the three generated alternative questions shown.
- (Optional) Source citations: Document IDs and source labels for each retrieved chunk.

Add Resume Tab

Candidates can submit their resume via form fields (Name, Email, Phone, Location, Job Category, Skills, Education, Summary, Experience). On submission, the record is saved, embedded, and immediately searchable.

Add Job Tab

Recruiters can post new job descriptions via form fields (Job Title, Job Description). On submission, the posting is saved, embedded, and immediately searchable.

VIII. References

1. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS), 33, 9459\u20139474. https://arxiv.org/abs/2005.11401
2. Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval, 3(4), 333\u2013389. https://doi.org/10.1561/1500000019
3. Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In Proceedings of the 32nd ACM SIGIR Conference (pp. 758\u2013759). https://doi.org/10.1145/1571941.1572114
4. Nogueira, R., & Cho, K. (2019). Passage Re-ranking with BERT. arXiv preprint arXiv:1901.04085. https://arxiv.org/abs/1901.04085
5. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In Proceedings of EMNLP-IJCNLP 2019, 3982\u20133992. https://arxiv.org/abs/1908.10084
6. Touvron, H., et al. (2023). Llama 2: Open Foundation and Fine-Tuned Chat Models. Meta AI Research. https://arxiv.org/abs/2307.09288
7. Grattafiori, A., et al. (2024). The Llama 3 Herd of Models. Meta AI Research. https://arxiv.org/abs/2407.21783
8. Costa-juss\u00e0, M. R., Cross, J., \u00c7elebi, O., Elbayad, M., Heafield, K., Heffernan, K., ... & Tran, C. (2022). No Language Left Behind: Scaling Human-Centered Machine Translation. Meta AI Research. https://arxiv.org/abs/2207.04672
9. Sneha Anbhavan (2022). Resume Dataset. Kaggle. https://www.kaggle.com/datasets/snehaanbhawal/resume-dataset
10. Kanchana (2023). Job Title and Description Dataset. Kaggle. https://www.kaggle.com/datasets/kanchana1990/job-description-dataset
11. Pinecone Documentation. (2024). Pinecone Serverless Vector Database. https://docs.pinecone.io

12. HuggingFace. (2024). Inference API Documentation. https://huggingface.co/docs/api-inference
13. Sentence Transformers. (2024). all-MiniLM-L6-v2 Model Card. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2
14. Sentence Transformers. (2024). paraphrase-multilingual-MiniLM-L12-v2 Model Card. https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
15. Hugging Face. (2024). cross-encoder/ms-marco-MiniLM-L-6-v2 Model Card. https://huggingface.co/cross-encoder/ms-marco-MiniLM-L-6-v2
16. Meta AI. (2024). facebook/nllb-200-distilled-600M Model Card. https://huggingface.co/facebook/nllb-200-distilled-600M
17. Streamlit Inc. (2024). Streamlit Documentation. https://docs.streamlit.io
18. Groq Inc. (2024). Groq API Documentation. https://console.groq.com/docs
19. Amazon Web Services. (2024). Amazon EC2 Documentation. https://docs.aws.amazon.com/ec2
20. Robertson, S. E., Walker, S., Jones, S., Hancock-Beaulieu, M., & Gatford, M. (1994). Okapi at TREC-3. In Proceedings of the Third Text Retrieval Conference (TREC-3), 109\u2013126.
21. Rank BM25 Python Library. (2023). rank_bm25: A Collection of BM25 Algorithms in Python. https://github.com/dorianbrown/rank_bm25
22. https://chatgpt.com/
23. https://claude.ai/new

IX. Appendix: Bonus Urdu Low-Resource Language Track

We built a parallel Urdu RAG pipeline with multilingual embeddings and a UI that accepts native Nastaliq script queries. The system implements a full hybrid retrieval pipeline, combining BM25, semantic embeddings, Reciprocal Rank Fusion (RRF), and Cross-Encoder re-ranking for Urdu content.
Key implementation details:

Data & Embeddings

The entire English corpus, consisting of 3,501 resumes and 63,763 job postings, was translated into Urdu. For generating embeddings in Urdu, the paraphrase-multilingual-MiniLM-L12-v2 model was used, which produces 384-dimensional embeddings and supports over 50 languages. Separate Pinecone indexes were created for storing the Urdu embeddings of candidates and job postings, named urdu-candidates and urdu-jobs. For BM25 retrieval, tokenization was performed using simple whitespace splitting, and no lowercasing was applied since Urdu script does not have letter casing.

Challenges & Solutions

- **Document Translation:** Document Translation for resumes is a tough task and after experimenting with multiple models and getting sub-par results we decided to stick to the one that gave the best results among those. However, the translations were still not very accurate.
- **Script handling:** Urdu Nastaliq is a right-to-left (RTL) script with complex ligatures. We added RTL CSS and Google Fonts (Noto Nastaliq Urdu) in the Streamlit UI to ensure correct rendering.
- **Translation artifacts:** Technical terms (e.g., “Python”) are often transliterated into Urdu. This was mitigated by keeping both English and Urdu fields, enabling hybrid cross-lingual matching.
- **Embedding model:** English-only models like MiniLM-L6-v2 produce meaningless embeddings for Urdu. Switched to paraphrase-multilingual-MiniLM-L12-v2, which supports Urdu natively.